

Real-Time Chat Application with AWS AppSync and DynamoDB

Project: Real-Time Chat Application with AWS AppSync and DynamoDB

Services: AWS AppSync, Amazon DynamoDB, Amazon Cognito, Amazon S3

Description: Create a cloud-based real-time chat application using AWS AppSync for GraphQL communication, DynamoDB for storing chat messages, Cognito for user authentication and management, and Amazon S3 for file uploads. The application enables users to send messages, upload files, and communicate through a modern web interface.

Skills: GraphQL, Real-time communication, Database management, Cloud storage, Authentication management, AWS cloud integration

Definition: The Real-Time Chat Application is a cloud-based messaging system designed using AWS serverless services. The application uses AWS AppSync to provide GraphQL APIs for communication between users and backend services. DynamoDB stores chat messages, Cognito manages user authentication and login functionality, and Amazon S3 stores uploaded files and media. The project demonstrates integration between multiple AWS services to build a scalable and modern cloud application.

Scope of the Project:

The scope of this project is to design and implement a fully functional cloud-based real-time chat application using AWS managed services.

This project covers:

- **Real-Time Messaging:** Users can send and receive chat messages using AWS AppSync GraphQL APIs.
- **Message Storage:** Chat messages are stored securely in Amazon DynamoDB.
- **User Authentication:** Amazon Cognito manages user signup and login functionality.
- **File Upload:** Users can upload files and images to Amazon S3.
- **Cloud-Based Deployment:** The frontend application is hosted using Amazon S3 Static Website Hosting.
- **GraphQL API Integration:** AppSync provides GraphQL queries and mutations for frontend communication.

Methodology:

1. AWS AppSync + DynamoDB + Cognito + S3 – Best Method

- Uses fully managed AWS serverless services.
- Provides GraphQL API-based communication.
- Enables scalable and modern cloud architecture.
- DynamoDB provides high-speed NoSQL database storage.
- Cognito securely handles user authentication.
- S3 enables reliable file upload and storage.
- Cost-effective and suitable for free-tier usage.
- Easy to deploy and manage without maintaining servers.

2. Traditional WebSocket + SQL Database – Alternative Method

- Uses WebSocket servers and relational databases.
- Requires backend server management.
- Higher operational complexity and maintenance.
- Requires manual scaling and infrastructure management.
- Less cost-efficient compared to AWS serverless architecture.
- Suitable for traditional application deployments.

AWS Services Used in the Project:

1. AWS AppSync

Definition: AWS AppSync is a managed GraphQL service that enables applications to securely access and manipulate data in real time.

Purpose: Provides GraphQL APIs for frontend communication.

Role:

- Handles GraphQL queries and mutations.
- Enables real-time messaging architecture.
- Connects frontend with DynamoDB.

2. Amazon DynamoDB

Definition: Amazon DynamoDB is a fully managed NoSQL database service that provides fast and scalable performance.

Purpose: Stores chat messages and application data.

Role:

- Stores usernames and messages.
- Retrieves chat history.
- Supports scalable database operations.

3. Amazon Cognito

Definition: Amazon Cognito is a user authentication and authorization service that helps manage user signup and login functionality.

Purpose: Provides secure user authentication.

Role:

- User registration and login.
- User management and authentication.
- Integrates with AppSync authorization.

4. Amazon S3 (Simple Storage Service)

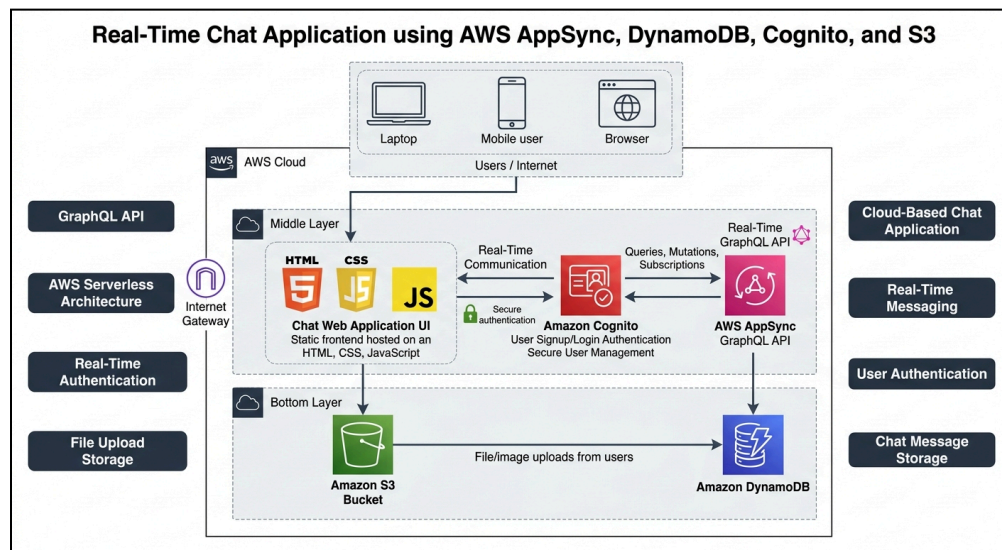
Definition: Amazon S3 is an object storage service used for storing files, images, and application assets.

Purpose: Stores uploaded files and frontend static website files.

Role:

- Stores uploaded images/files.
- Hosts frontend application using static website hosting.
- Provides scalable cloud storage.

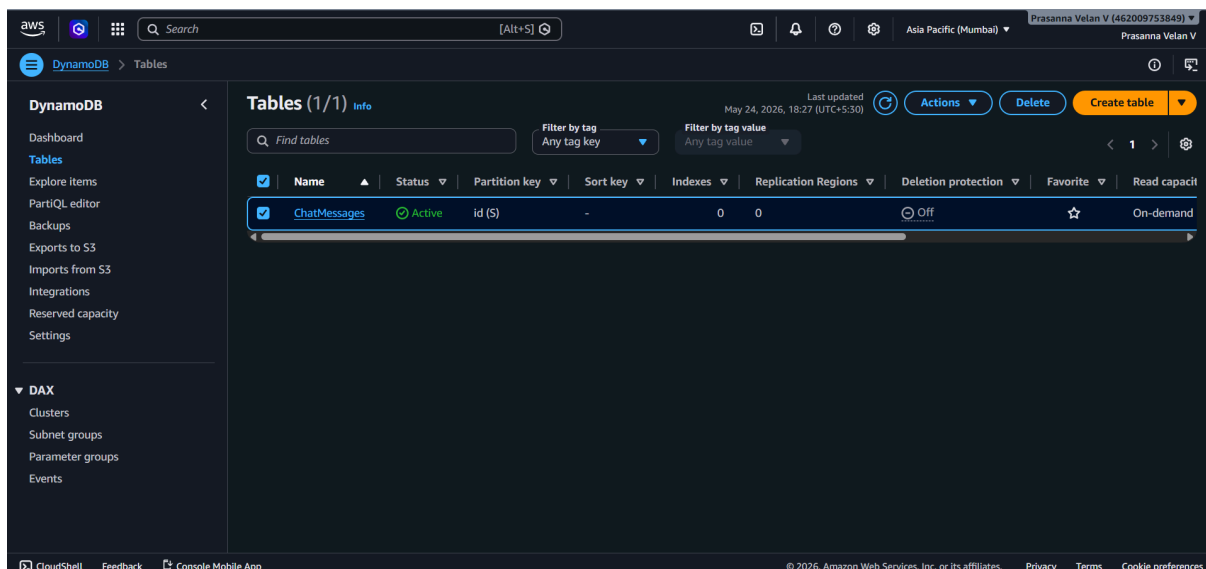
Architecture Diagram:



Implementation Steps to Do the Project:

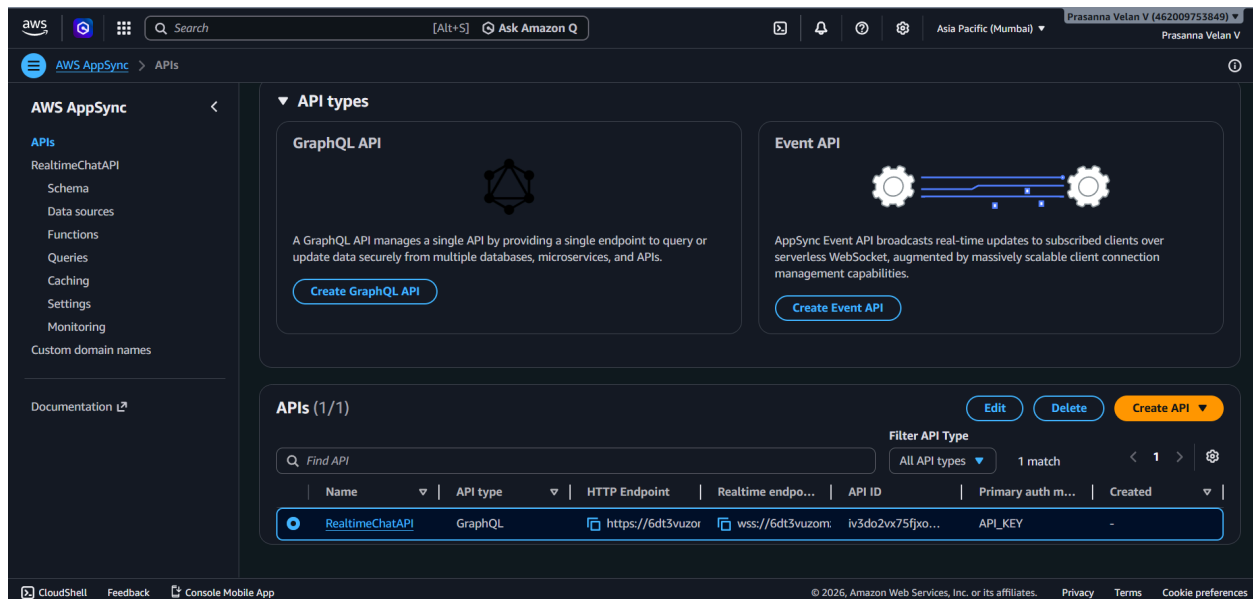
Step 1: Create DynamoDB Table

- Go to AWS Console → DynamoDB.
- Click Create Table.
- Table Name: ChatMessages
- Partition Key: id (String)
- Create the table.



Step 2: Create AppSync GraphQL API

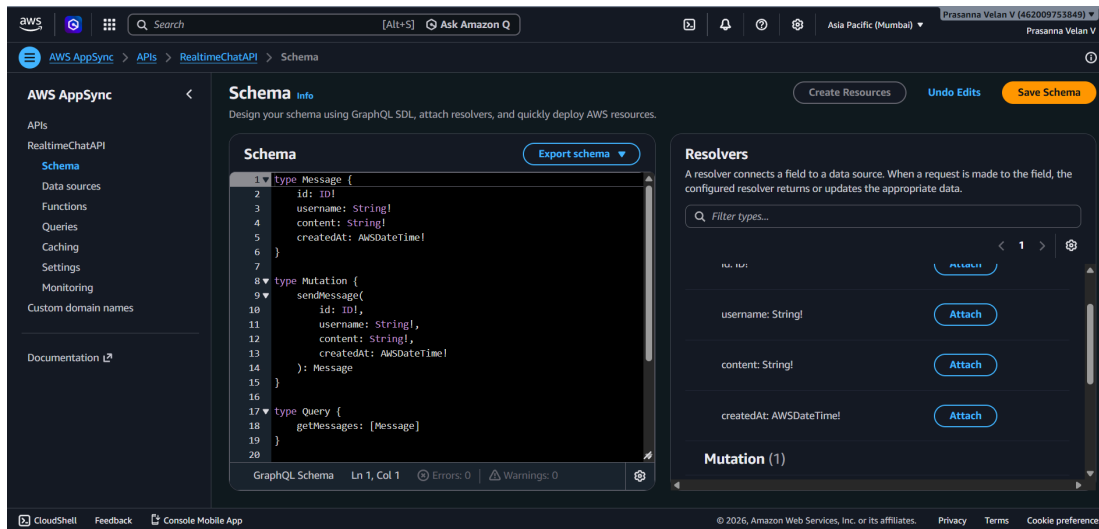
- Go to AWS Console → AppSync.
- Click Create API.
- Select GraphQL API.
- API Name: RealtimeChatAPI
- Authorization Type: API Key
- Create API.



Step 3: Configure GraphQL Schema

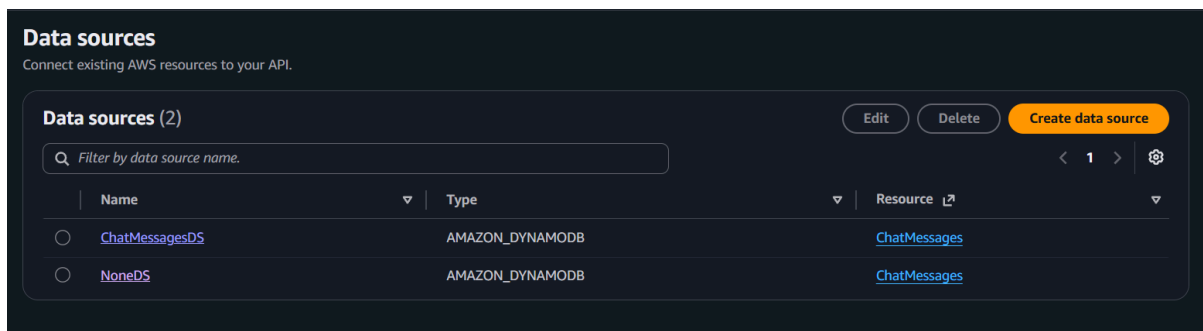
- Go to Schema section.
- Create Message type.
- Create Query, Mutation, and Subscription schema.
- Save schema.

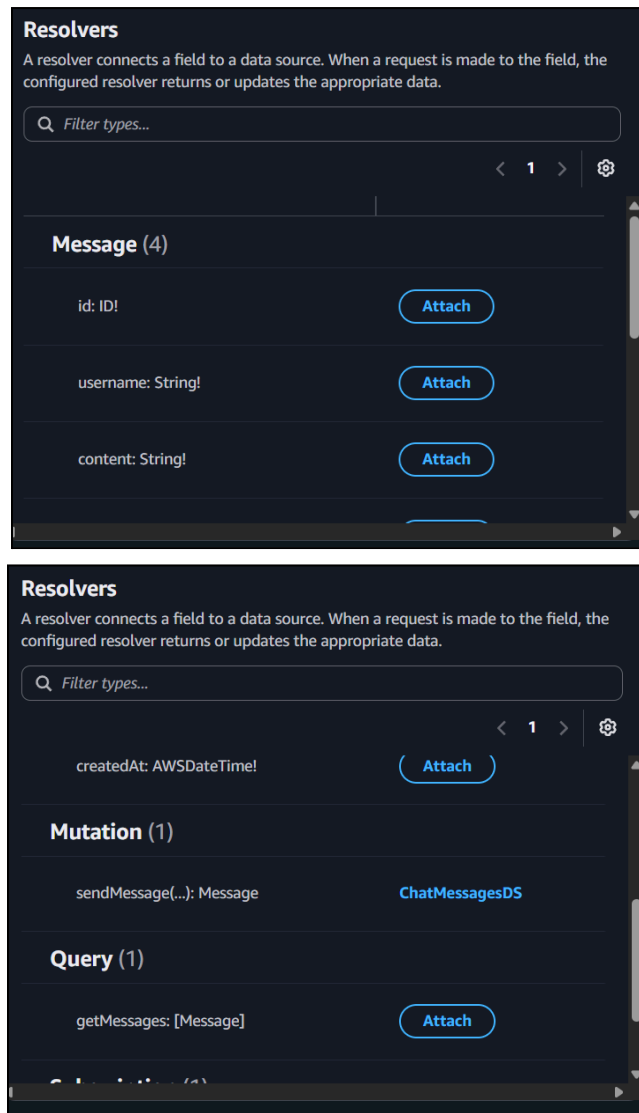
Real-Time Chat Application with AWS AppSync and DynamoDB



Step 4: Configure DynamoDB Resolver

- Create DynamoDB data source.
- Connect ChatMessages table.
- Attach resolver for sendMessage mutation.
- Attach resolver for getMessages query.
- Save resolvers.





Step 5: Test GraphQL API

- Open Queries section.
- Run GraphQL mutation.
- Verify messages are stored in DynamoDB.
- Run query to fetch messages.

Real-Time Chat Application with AWS AppSync and DynamoDB

The screenshot shows the AWS AppSync console interface. On the left, a sidebar lists navigation options: APIs, RealtimeChatAPI, Schema, Data sources, Functions, **Queries**, Caching, Settings, Monitoring, and Custom domain names. The main area is titled 'Queries' with the subtitle 'Write, validate, and test GraphQL queries.' It features a code editor with a query:

```
1 query {
2   getMessages {
3     id
4     username
5     content
6     createdAt
7   }
8 }
```

 An 'Execute query (Ctrl-Enter)' button is visible. Below the editor is a 'Variables' section with a single entry:

```
1 {}
```

. On the right, the JSON response is displayed:

```
{
  "data": {
    "getMessages": [
      {
        "id": "1779625427136",
        "username": "prasanna_velan_",
        "content": "",
        "createdAt": "2026-05-24T12:23:47.136Z"
      },
      {
        "id": "1779625450743",
        "username": "prasanna",
        "content": "vanakkam",
        "createdAt": "2026-05-24T12:24:10.743Z"
      },
      {
        "id": "1779626459923",
        "username": "prasanna",
        "content": "this is my image",
        "createdAt": "2026-05-24T12:40:59.923Z"
      }
    ]
  }
}
```

The screenshot shows the AWS AppSync console interface. On the left, a sidebar lists navigation options: APIs, RealtimeChatAPI, Schema, Data sources, Functions, **Queries**, Caching, Settings, Monitoring, and Custom domain names. The main area is titled 'Queries' with the subtitle 'Write, validate, and test GraphQL queries.' It features a code editor with a mutation:

```
1 mutation {
2   sendMessage(
3     id: "2",
4     username: "Prasanna",
5     content: "Realtime working!",
6     createdAt: "2026-05-24T10:10:00Z"
7   ) {
8     id
9     username
10    content
11  }
12 }
```

 An 'Execute query (Ctrl-Enter)' button is visible. Below the editor is a 'Variables' section with a single entry:

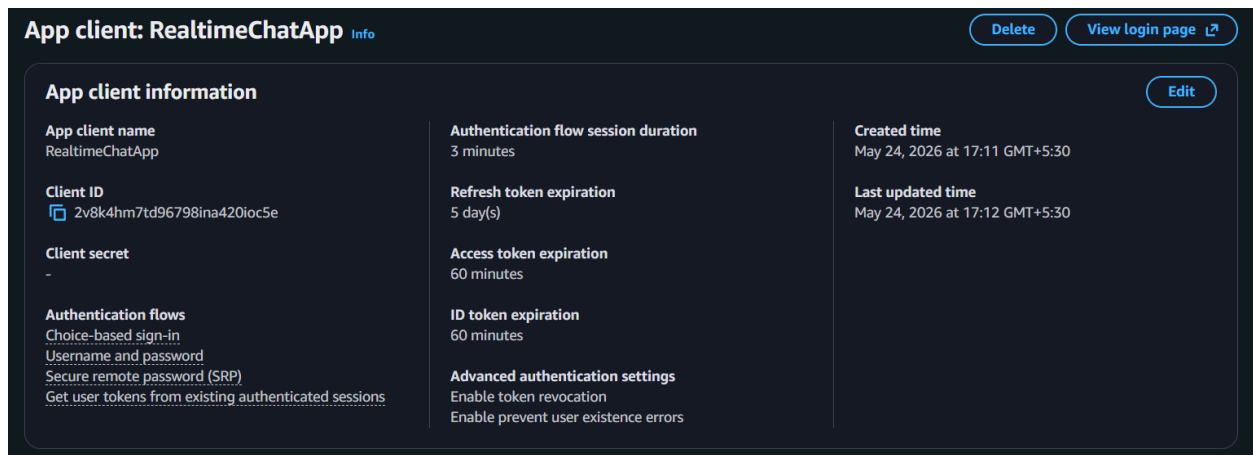
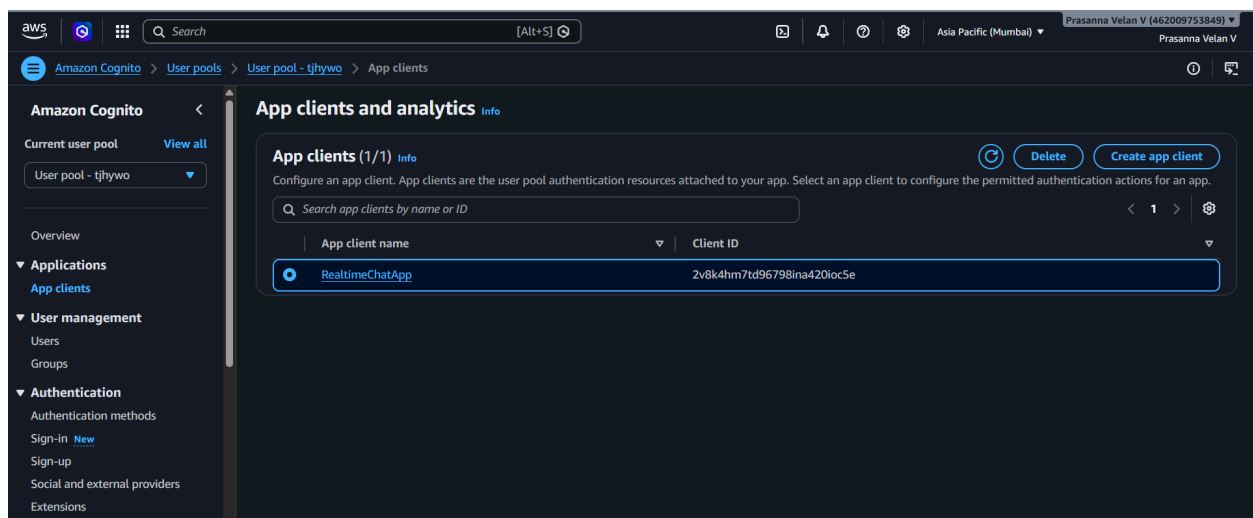
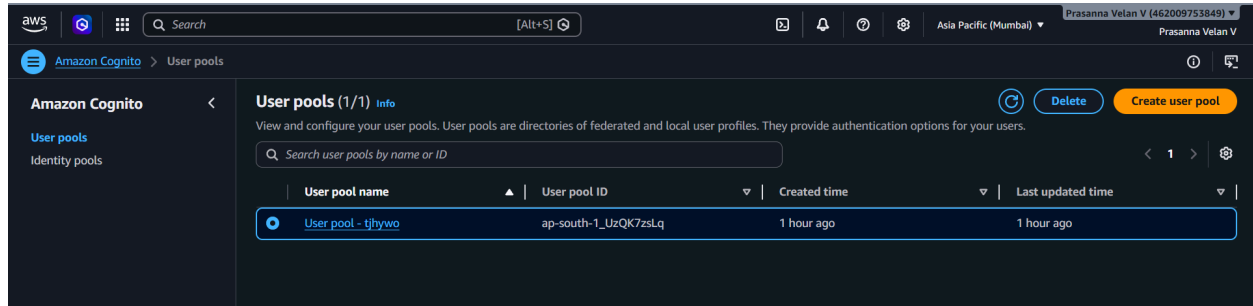
```
1 {}
```

. On the right, the JSON response is displayed:

```
{
  "data": {
    "sendMessage": {
      "id": "2",
      "username": "Prasanna",
      "content": "Realtime working!"
    }
  }
}
```

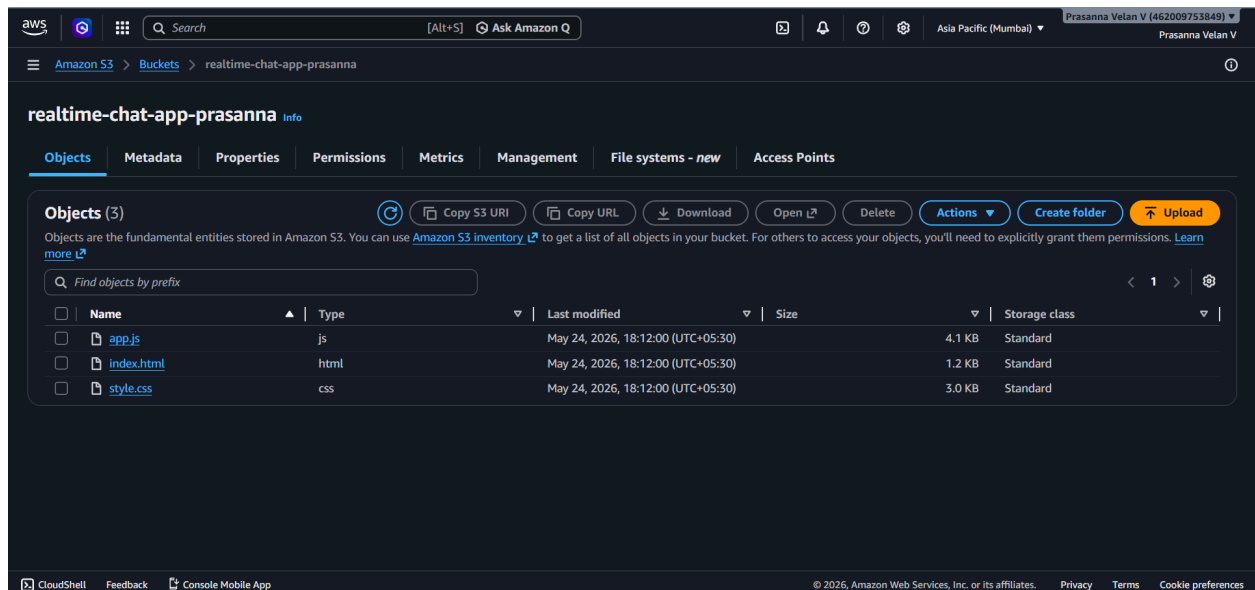
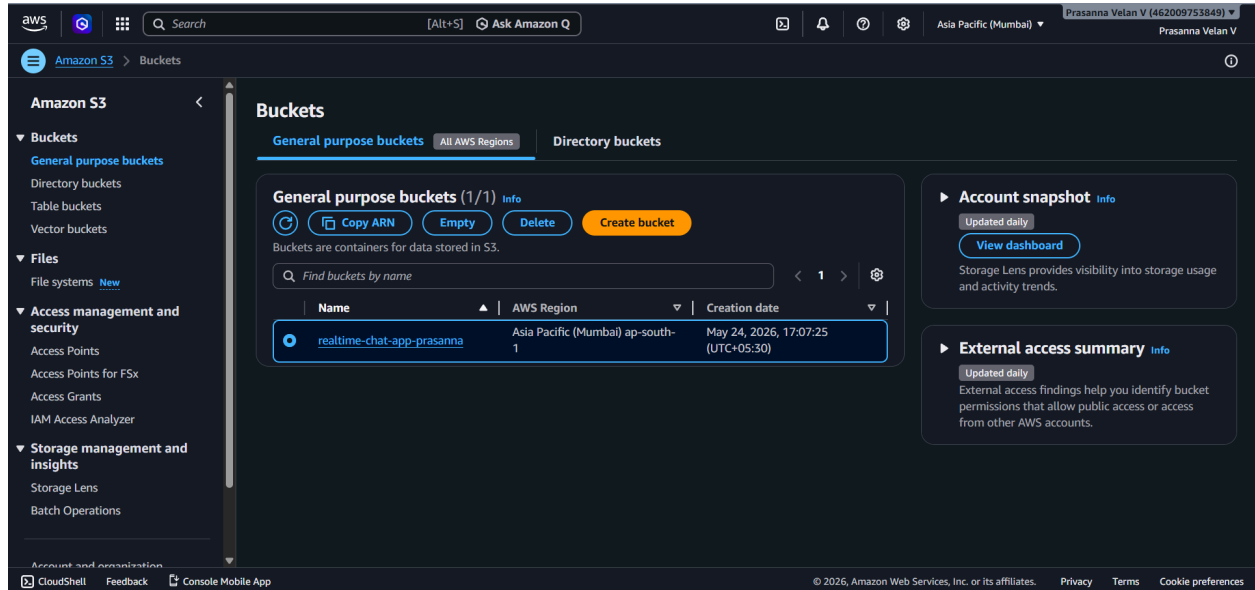

Step 6: Create Cognito User Pool

- Go to Cognito Console.
- Create User Pool.
- Configure email authentication.
- Create App Client.
- Save User Pool ID and Client ID.



Step 7: Configure S3 Bucket

- Create S3 bucket.
- Enable Static Website Hosting.
- Configure bucket permissions.
- Add CORS configuration.
- Upload frontend files.

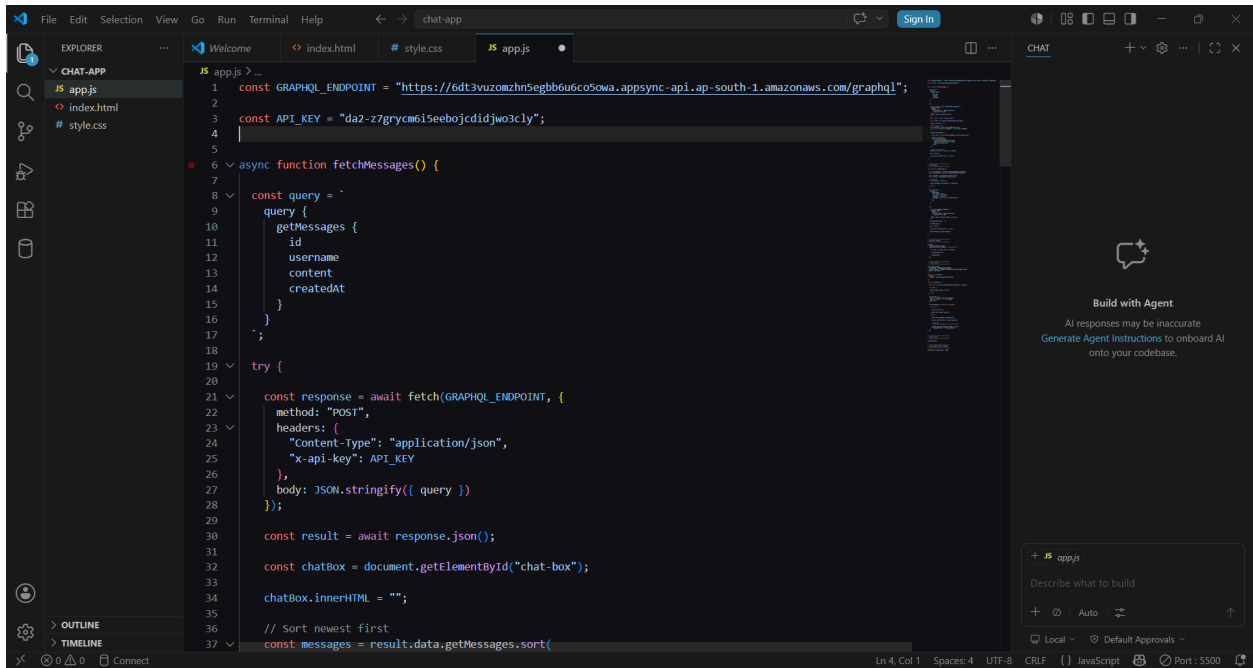


Step 8: Develop Frontend Application

Create: index.html ,style.css, app.js

Integrate:

- AppSync GraphQL API
- S3 file upload
- Cognito authentication
- DynamoDB message retrieval

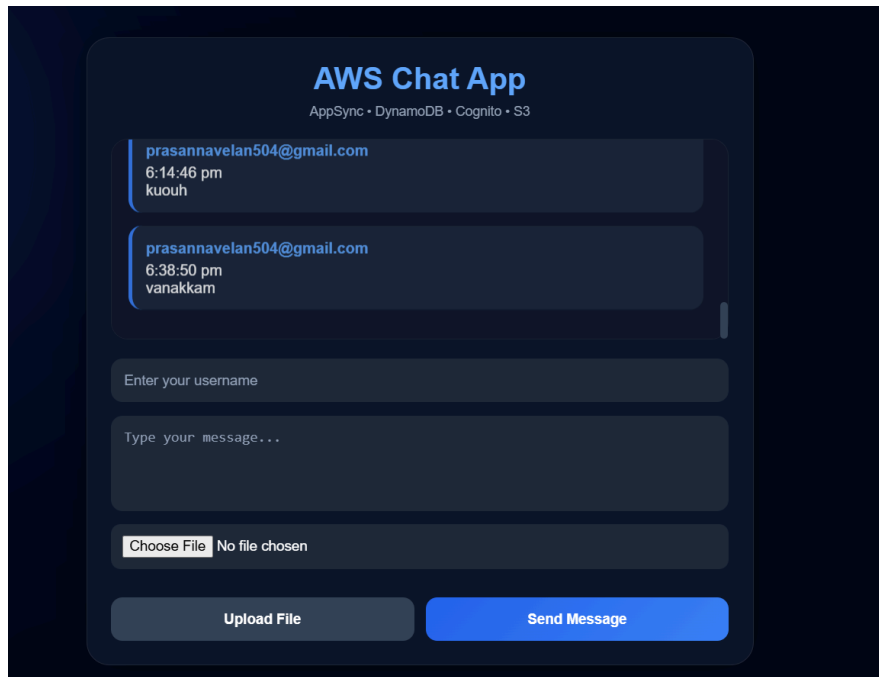


```
1  const GRAPHQL_ENDPOINT = "https://edt3vuzomzhn5egbbu6co5owa.appsync-api.ap-south-1.amazonaws.com/graphql";
2
3  const API_KEY = "da2-z7grycm61seebojcdidjwo3cly";
4
5
6  async function fetchMessages() {
7
8      const query = `
9      query {
10        getMessages {
11          id
12          username
13          content
14          createdAt
15        }
16      }`;
17
18
19      try {
20
21          const response = await fetch(GRAPHQL_ENDPOINT, {
22            method: "POST",
23            headers: {
24              "Content-Type": "application/json",
25              "x-api-key": API_KEY
26            },
27            body: JSON.stringify({ query })
28          });
29
30          const result = await response.json();
31
32          const chatBox = document.getElementById("chat-box");
33
34          chatBox.innerHTML = "";
35
36          // Sort newest first
37          const messages = result.data.getMessages.sort(
```

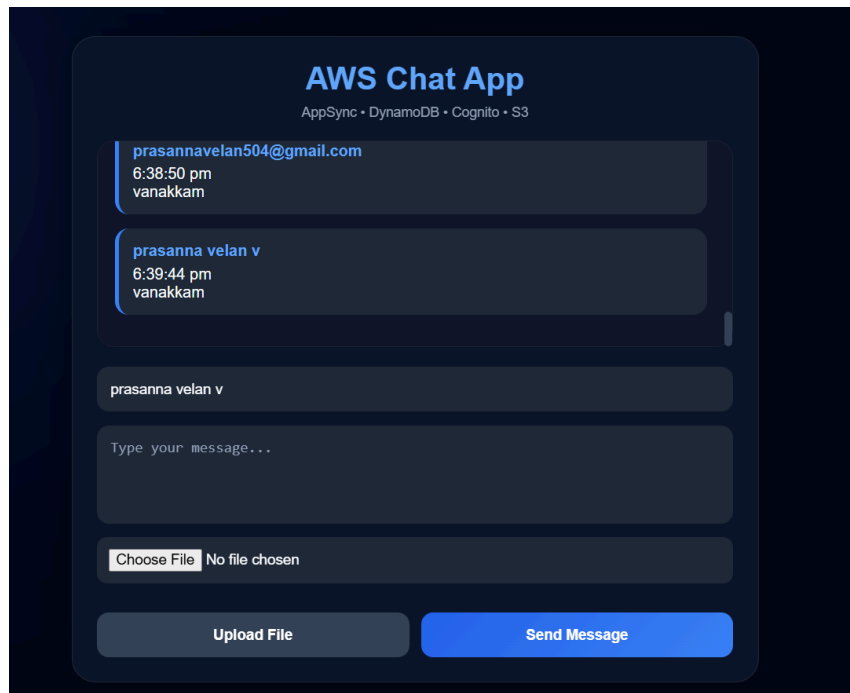
Step 9: Test Chat Application

- Open frontend application.
- Send messages.
- Upload files.
- Verify data in DynamoDB and S3.
- Confirm GraphQL communication.

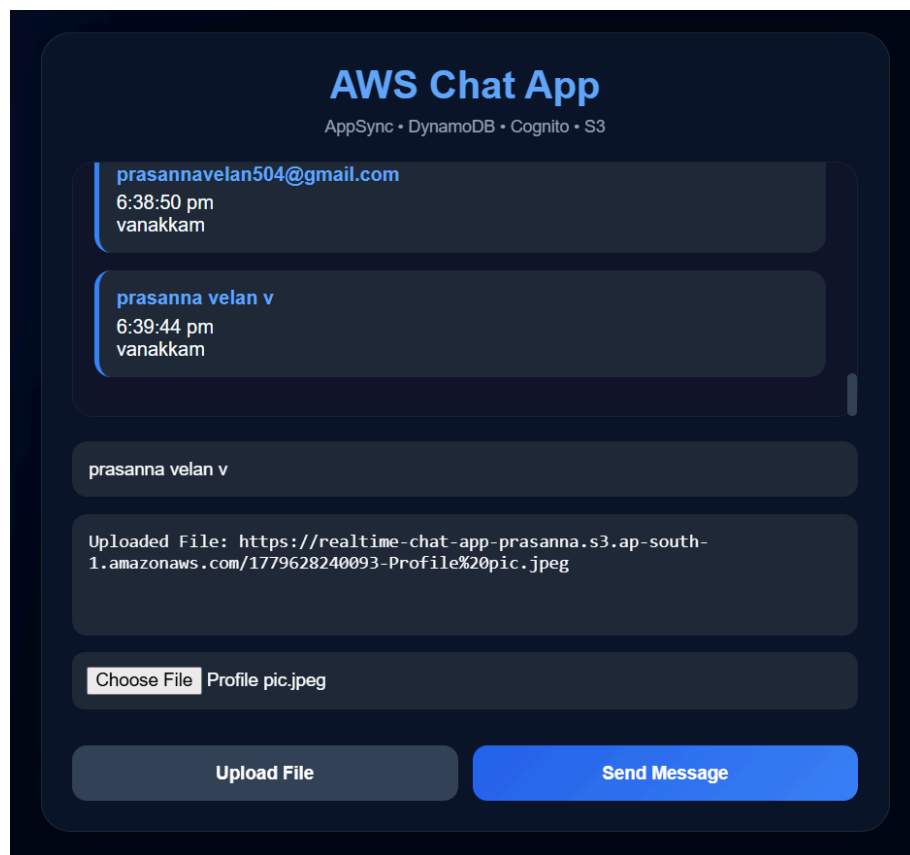
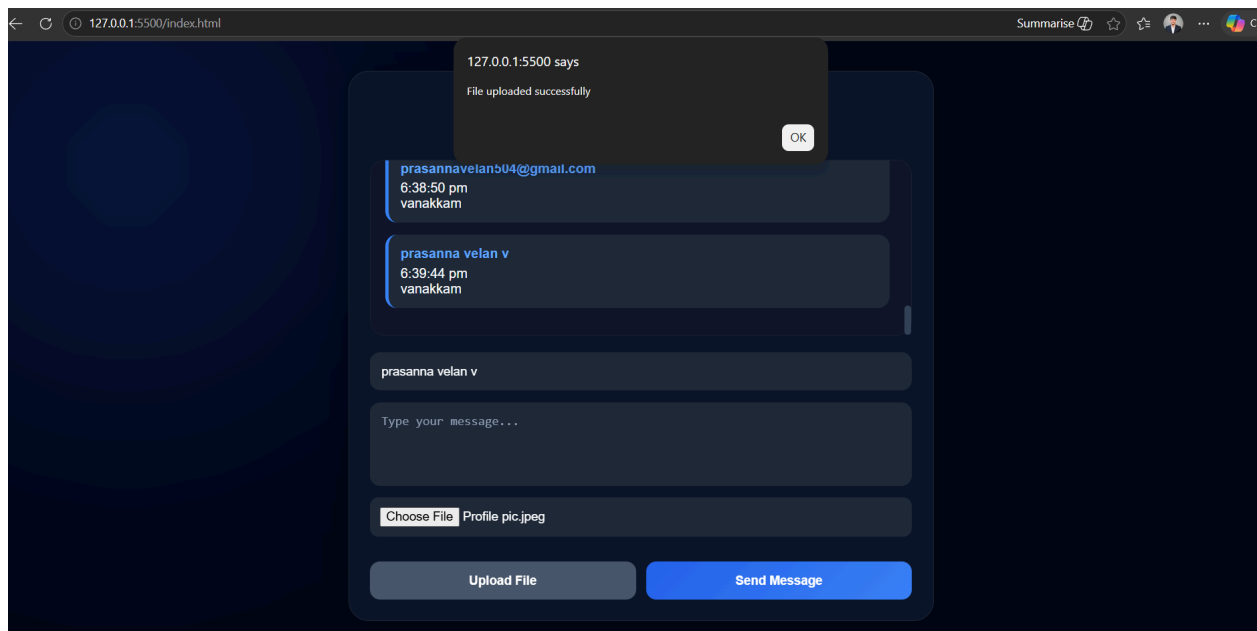
Real-Time Chat Application with AWS AppSync and DynamoDB



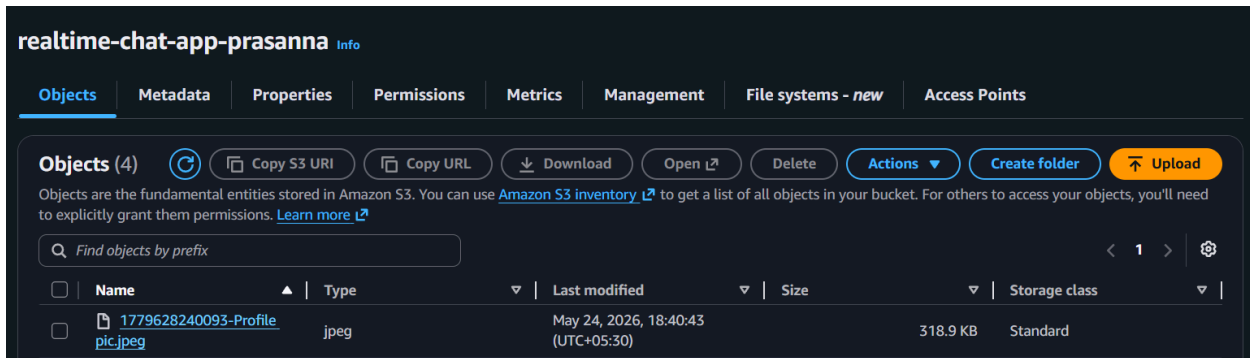
Message:



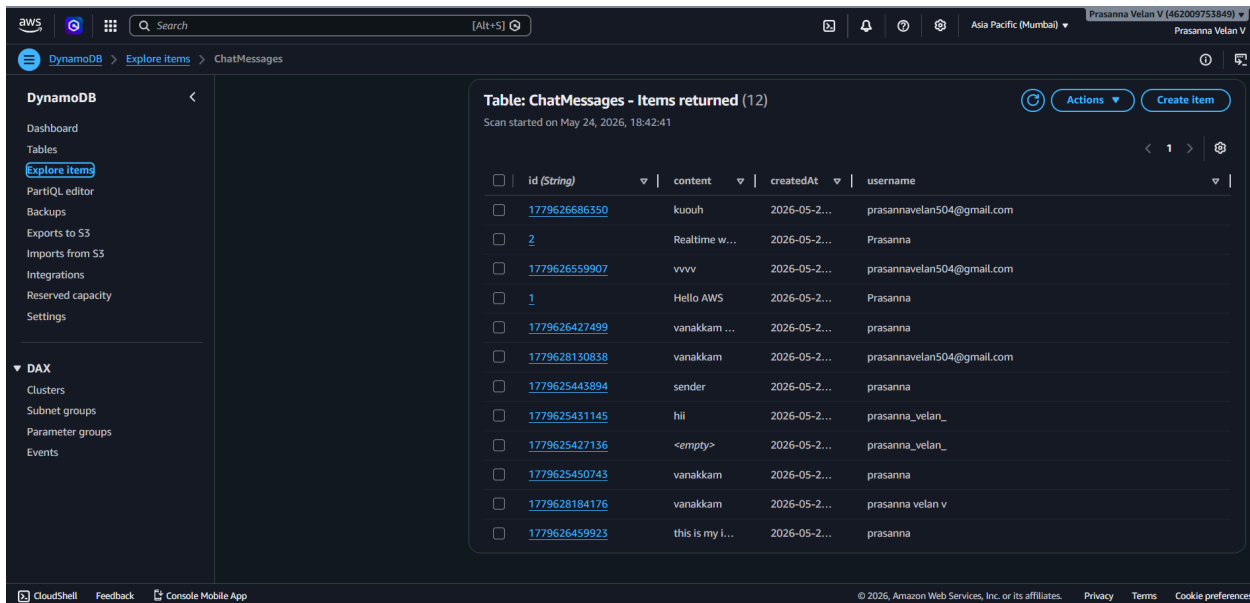
Upload Image/Video:



Inside S3:



Inside Db:



Troubleshooting:

Issue 1: GraphQL Mutation Failed

The AppSync resolver configuration was incorrect or missing.

Solution:

- Verify GraphQL schema.
- Reattach DynamoDB resolver.
- Save and deploy schema properly.

Issue 2: File Upload Failed in S3

CORS or IAM permissions were not configured correctly.

Solution:

- Configure S3 CORS policy.
- Attach AmazonS3FullAccess policy to IAM user.
- Verify bucket permissions and public access settings.

Issue 3: API Authorization Error

The AppSync API key or Cognito configuration was invalid.

Solution:

- Verify AppSync API Key.
- Reconfigure authorization mode.
- Check Cognito User Pool integration.

Conclusion:

The Real-Time Chat Application provides a scalable and cloud-native messaging platform using AWS managed services. AWS AppSync enables GraphQL-based communication between frontend and backend services, while DynamoDB securely stores chat messages. Amazon Cognito manages user authentication and Amazon S3 handles file uploads and static website hosting. The application demonstrates integration of multiple AWS serverless services to build a modern real-time cloud application. The solution is scalable, cost-effective, easy to deploy, and suitable for cloud computing projects and organizational use cases.